

BÁO CÁO KẾT QUẢ THỰC HÀNH

Môn học: Java Web Service

Chủ đề: Xây dựng hệ thống MediCore quản lý y tế và lịch hẹn khám bệnh

Họ và tên sinh viên: An Hải Dũng

Mã sinh viên: B24DTCN200

Lớp: HN-KS24-CNTT2

I. Nội dung thực hành

Trong buổi thực hành ss21, em xây dựng project MediCore theo mô hình REST API bằng Spring Boot. Hệ thống tập trung vào quản lý thông tin bệnh nhân, bác sĩ, thuốc, lịch hẹn khám bệnh, đồng thời bổ sung chức năng xác thực bằng JWT, refresh token, đăng xuất và phân quyền theo vai trò người dùng.

1. Kiến trúc hệ thống

Project được tổ chức theo các package chính sau:

- controller: AuthController, PatientController, DoctorController, MedicineController và AppointmentController tiếp nhận request HTTP, gọi service xử lý nghiệp vụ và trả về ResponseEntity kèm ApiDataResponse.
- service: AuthService, PatientService, DoctorService, MedicineService và AppointmentService triển khai logic xác thực, CRUD dữ liệu và đặt lịch hẹn.
- repository: các repository kế thừa JpaRepository để thao tác với MySQL như UserRepository, RoleRepository, PatientRepository, DoctorRepository, MedicineRepository và AppointmentRepository.
- entity: định nghĩa các bảng chính gồm User, Role, Patient, Doctor, Medicine, Appointment, TokenSession, RevokedToken, Prescription và PrescriptionItem.
- dto: tách request và response thành các lớp DTO như LoginRequest, RegisterRequest, PatientRequestDTO, DoctorRequestDTO, MedicineRequestDTO, AppointmentRequestDTO và các ResponseDTO tương ứng.
- security: cấu hình Spring Security, JwtService, JwtAuthenticationFilter, JwtAuthenticationEntryPoint và CustomUserDetailsService để bảo vệ API bằng JWT.
- exception: xây dựng GlobalExceptionHandler, ApiException, ResourceNotFoundException, UnauthorizedException và ForbiddenException để xử lý lỗi thống nhất.
- config và aspect: DataInit tạo dữ liệu mẫu ban đầu; LoggingAspect hỗ trợ ghi log quá trình gọi service.

2. Các công nghệ và kỹ thuật áp dụng

- Spring Boot: khởi tạo và chạy ứng dụng backend.
- Spring Web MVC REST: xây dựng các API GET, POST, PUT và DELETE.
- Spring Data JPA: làm việc với database thông qua JpaRepository.
- MySQL Connector/J: kết nối ứng dụng với hệ quản trị cơ sở dữ liệu MySQL.

- Spring Security: bảo vệ API, kiểm tra JWT và phân quyền bằng `@PreAuthorize`.
- JWT: tạo, ký, kiểm tra access token và refresh token.
- BCryptPasswordEncoder: mã hóa mật khẩu trước khi lưu vào database.
- Jakarta Validation: kiểm tra dữ liệu đầu vào bằng `@NotBlank`, `@NotNull`, `@Positive` và `@PositiveOrZero`.
- Lombok: giảm code getter, setter, constructor và builder.
- AOP Logging: ghi log khi gọi các service trong hệ thống.
- JUnit, Mockito và Jacoco: phục vụ kiểm thử service và đo coverage.

3. Xử lý logic và nghiệp vụ

- Đăng ký tài khoản: nhận username, password, email và fullName; kiểm tra username tồn tại, mã hóa mật khẩu, gán `ROLE_PATIENT` và tạo hồ sơ Patient tương ứng.
- Đăng nhập: xác thực username và password, sinh access token và refresh token, sau đó lưu refresh token vào bảng `token_sessions`.
- Refresh token: kiểm tra refresh token chưa revoked và chưa expired, sau đó cấp access token mới.
- Đăng xuất: đánh dấu các refresh token của user là revoked/expired và lưu access token vào bảng `revoked_tokens` để blacklist.
- Quản lý bệnh nhân: admin có thể xem danh sách, xem chi tiết, thêm, cập nhật và xóa bệnh nhân.
- Quản lý bác sĩ: admin có thể CRUD thông tin bác sĩ gồm họ tên, chuyên khoa, số điện thoại, email và phòng khám.
- Quản lý thuốc: admin có thể CRUD thuốc gồm tên, mô tả, giá và số lượng tồn kho.
- Đặt lịch hẹn: bệnh nhân đã đăng nhập chọn doctorId và appointmentTime, hệ thống tạo Appointment với trạng thái BOOKED.
- Xem lịch hẹn cá nhân: bệnh nhân chỉ xem lịch hẹn gắn với tài khoản hiện tại.
- Xem lịch hẹn hôm nay của bác sĩ: bác sĩ chỉ xem lịch trong ngày của chính mình dựa trên Doctor profile liên kết với user.
- Admin xem toàn bộ lịch hẹn: phục vụ quản lý tổng thể trong hệ thống.
- Xử lý lỗi: nếu không tìm thấy user, doctor, patient hoặc medicine thì ném `ResourceNotFoundException`; lỗi xác thực dùng `UnauthorizedException`.

II. Kiểm thử và triển khai

Ứng dụng được cấu hình bằng Gradle, sử dụng Java 17, Spring Boot và MySQL. Database được cấu hình trong `application.properties` với tên MediCore, Hibernate `ddl-auto=update` để tự cập nhật cấu trúc bảng theo entity. Các API có thể kiểm thử bằng Postman collection đi kèm project.

Các endpoint chính:

- POST `/api/auth/register`: đăng ký tài khoản bệnh nhân.
- POST `/api/auth/login`: đăng nhập và nhận access token, refresh token.
- POST `/api/auth/refresh`: làm mới access token.

- POST /api/auth/logout: đăng xuất và thu hồi token.
- GET /api/patients: admin lấy danh sách bệnh nhân.
- GET /api/patients/{id}: admin lấy chi tiết bệnh nhân.
- POST /api/patients: admin thêm bệnh nhân.
- PUT /api/patients/{id}: admin cập nhật bệnh nhân.
- DELETE /api/patients/{id}: admin xóa bệnh nhân.
- GET /api/doctors: admin lấy danh sách bác sĩ.
- POST /api/doctors: admin thêm bác sĩ.
- PUT /api/doctors/{id}: admin cập nhật bác sĩ.
- DELETE /api/doctors/{id}: admin xóa bác sĩ.
- GET /api/medicines: admin lấy danh sách thuốc.
- POST /api/medicines: admin thêm thuốc.
- PUT /api/medicines/{id}: admin cập nhật thuốc.
- DELETE /api/medicines/{id}: admin xóa thuốc.
- POST /api/appointments: bệnh nhân đặt lịch hẹn.
- GET /api/appointments/my: bệnh nhân xem lịch hẹn của mình.
- GET /api/appointments/doctor/today: bác sĩ xem lịch hẹn trong ngày.
- GET /api/appointments: admin xem toàn bộ lịch hẹn.

III. Công việc đã thực hiện

1. Công việc cá nhân

- Xây dựng các entity chính cho hệ thống MediCore như User, Role, Patient, Doctor, Medicine, Appointment, TokenSession và RevokedToken.
- Xây dựng các DTO request/response để nhận dữ liệu từ client và trả dữ liệu thống nhất.
- Xây dựng controller cho các nhóm chức năng auth, bệnh nhân, bác sĩ, thuốc và lịch hẹn.
- Xây dựng service xử lý nghiệp vụ CRUD, đăng ký, đăng nhập, refresh token, logout và đặt lịch hẹn.
- Cấu hình Spring Security để permitAll cho /api/auth/** và yêu cầu JWT cho các API còn lại.
- Áp dụng phân quyền theo role ADMIN, DOCTOR và PATIENT bằng @PreAuthorize.
- Thiết kế ApiDataResponse để response trả về thống nhất gồm success, message, data và status.
- Tạo dữ liệu mẫu ban đầu bằng DataInit gồm admin, doctor, patient và một số thuốc.
- Viết test cho các service chính và cấu hình Jacoco để đo độ bao phủ kiểm thử.

2. Công việc nhóm

- Thống nhất cấu trúc package theo mô hình Controller - Service - Repository - Entity - DTO.
- Phối hợp kiểm thử API bằng Postman và xử lý lỗi khi request không hợp lệ hoặc dữ liệu không tồn tại.

- Thống nhất cách đặt tên endpoint, response message và phân quyền cho từng nhóm người dùng.

IV. Khó khăn

- Cần hiểu rõ luồng hoạt động của Spring Security khi kết hợp JWT filter với SecurityContext.
- Cần phân biệt access token và refresh token, đồng thời xử lý trường hợp token bị revoked hoặc expired.
- Cần thiết kế quan hệ giữa User với Role, Patient, Doctor và TokenSession sao cho phù hợp nghiệp vụ.
- Cần lấy đúng user hiện tại từ SecurityContext để bệnh nhân chỉ xem lịch của mình và bác sĩ chỉ xem lịch của mình.
- Cần xử lý exception thống nhất để client nhận được thông báo lỗi rõ ràng.
- Cần kiểm tra role khi gọi API để tránh người dùng truy cập sai chức năng.

V. Kết quả đạt được

- Xây dựng thành công REST API cho hệ thống MediCore quản lý bệnh nhân, bác sĩ, thuốc và lịch hẹn khám bệnh.
 - Hoàn thiện chức năng đăng ký, đăng nhập, refresh token, logout và blacklist token khi đăng xuất.
 - Hệ thống hỗ trợ phân quyền theo vai trò ADMIN, DOCTOR và PATIENT.
 - Các API trả về response thống nhất, giúp client dễ đọc trạng thái xử lý.
 - Database MySQL được ánh xạ bằng JPA entity, hỗ trợ tự cập nhật schema bằng Hibernate.
 - Project có dữ liệu mẫu ban đầu và test service để hỗ trợ kiểm thử.
- GitHub: https://github.com/AnHaiDung/Java_Web_SS29_mini_project

VI. Kinh nghiệm rút ra

- Tách Controller, Service, Repository, Entity và DTO giúp code rõ trách nhiệm và dễ bảo trì.
- Khi làm API có xác thực, cần thiết kế luồng token rõ ràng từ đăng nhập, refresh đến đăng xuất.
- Phân quyền bằng role giúp hệ thống an toàn hơn và hạn chế truy cập sai chức năng.
- Validation ở DTO giúp phát hiện dữ liệu sai trước khi đi vào tầng xử lý nghiệp vụ.
- Response thống nhất giúp frontend hoặc Postman dễ kiểm tra kết quả trả về.
- Việc viết test service giúp kiểm tra logic nghiệp vụ độc lập với controller và database thật.